

Anchor.

Reliable Memory for Agents

面向 Agent 的可信长期记忆
零显式反馈闭环实现与初步评测

摘要

长期记忆让 Agent 可以跨任务复用事实和经验，也会把一次性要求带到不相关的未来任务。本报告研究一个工程问题：用户没有对 Memory 打分、点赞或主动纠错时，系统能否依靠正常业务证据发现记忆问题，进行局部、可追溯的处理，并以较低推理成本验证后续任务质量。

我们在 TencentDB Agent Memory 的 MemoryCore 上实现 Anchor。系统保留 L0 原始来源，在 L1 候选写入前核验内容与作用范围，dedup 后再次审核最终草稿；读取侧记录精确的 `recordId + version`、真实 prompt 暴露及当前任务工具回执。证据不足时返回 `unknown` 或进入 review queue。对于“业务事实与本次回复格式混在同一候选”的常见弱模型错误，Anchor 从用户原文投影可独立成立的事实句，在线处理增加 0 次模型调用。

初步结果显示：最初词法 baseline 在 LongMemEval 上的反馈精度为 43.33%，产生 34 次错误失效；两个新业务来源的 Qwen3-4B 端到端实验中，strict exact 从普通臂 2/4 提升到调整臂 4/4，业务事实均为 4/4，严重回归为 0，全生命周期 token 下降 2.98%。新增混合主张分支在 14 个新模板上实现动作 14/14、内容 14/14，并在一条历史真实 Qwen 候选重放中将格式残留从 1 降至 0，同时保留 3/3 业务事实。

这些结果证明了开发初版的可行性。端到端来源只有两个，固定模型和种子也限制了统计外推；当前结论用于工程验收和后续实验设计，不构成商业系统排行。

开源实现

[github.com/newmancai/TencentDB-Agent-Memory · feat/anchor-memory](https://github.com/newmancai/TencentDB-Agent-Memory/blob/main/feat/anchor-memory)

实现基准提交：69486006e02800ad1a686399b4c0a6c9bf99acb70

1. 研究问题与验收标准

1.1 问题定义

Agent Memory 的写入通常由模型完成。模型能够压缩对话，也可能把局部指令泛化成长期规则。例如，用户同时描述生产限制、试点有效期，并要求“本工单按固定模板回复”。生产限制和有效期具有跨任务价值；回复模板只对当前工单有效。两者被写进同一条 active L1 后，后续任务会读到正确事实，同时受过期行为约束干扰。

本项目关注三类风险：

1. 内容风险：候选加入了来源没有支持的事实，或在改写中改变了对象、状态、时间与语言。
2. 范围风险：session、ticket 或当天有效的要求被提升为长期行为。
3. 归因风险：历史工具回执、模型自述或局部 API 成功被误当成当前任务完成，继而错误奖惩 Memory。

1.2 零显式反馈边界

运行时允许使用正常业务对话、真实工具 call/result、数据库或文件状态、业务不变量。用户对 Memory 的纠错标签、满意度评分、点赞、隐藏答案和离线 grader 不参与当次记忆决策。离线 grader 只衡量最终效果。

验收同时观察四个维度：

维度	核心问题	指标
判断可靠性	找到的问题是否真实存在	feedback precision/recall、 validity、 unknown
生命周期安全	有害行为是否退出 active 集合，有用事实是否保留	false activation、 false invalidation、 useful retention
后续任务质量	处理后任务是否改善或保持	strict exact、 semantic facts、 severe regression
AI Infra 成本	获取反馈和处理记忆需要多少资源	模型调用、 tokens、 延迟、 升级率

2. Anchor 的设计

2.1 三个设计原则

来源可回查。L0 原始消息保留完整顺序；L1 候选携带 `source_message_ids`。系统需要说明每个长期主张来自哪条用户消息。

保存价值与行为效力分开。临时事件可以留在来源层供搜索，长期 active instruction 需要更强的范围证据。隔离行为不等于删除历史。

当前任务证据优先。召回、prompt 暴露、工具执行和最终输出分别记录。只有属于当前 `taskRunId` 的客观终态，才具备进入 Memory 归因研究的资格。

2.2 写入链路

```
业务对话
├─ L0: 保存有序原始消息和 message ID
└─ Qwen L1 extractor
    ├─ evidence-scoped admission
    │   └─ 检查说话人、来源支持、语言和临时/长期范围
    ├─ MemoryCore dedup
    └─ final-draft admission
        ├─ retain      写入 active L1
        ├─ project     从原文复制独立事实句
        ├─ quarantine  保留 L0, 加入 review queue
        └─ deferred    暂缓复杂 update / merge
```

dedup 后的二次审核很关键。候选最初合格，并不保证被去重或合并后的最终文字、类型和动作仍受原始证据支持。

2.3 读取、反馈与局部干预

```
auto-recall / tdai_memory_search
→ 记录 recordId、version、rank、score、预算和截断
→ host 确认内容进入真实 prompt，标记 exposed
→ 工具 call/result 绑定当前 taskRunId
→ objective validators 产生 success / failure / unknown
→ 定位 retrieval / exposure / execution / downstream
→ 精确屏蔽目标 recordId + version
→ 相同配置执行 unadjusted / adjusted 两臂
```

自动召回和显式 Memory Search 共用精确排除集合，避免同一记录从旁路重新进入上下文。上一任务成功、工具名出现或模型声称“已完成”都不能替代当前任务回执。

2.4 混合主张投影

Anchor 针对高频、证据清楚的范围错误采用确定性处理：

1. 根据 `source_message_ids` 取回原始用户消息；
2. 找到“仅本次、当前工单、今天”等局部范围及回复行为；
3. 原文业务事实可以独立成句时，直接复制事实句作为 episodic；
4. 事实与临时规则无法安全分句时，候选进入 quarantine；
5. “API 输出格式已改为 JSON”这类业务事实保持原样；
6. 跨句推断、冲突消解和复杂更新留给异步 reviewer。

投影使用原文，降低规则系统自行改写事实的风险。在线路径不调用模型；保守 quarantine 可能损失一部分召回，后续需要测量 review queue 的补回能力。

3. 实现范围

Anchor 复用 MemoryCore 现有 L0/L1、dedup、检索和宿主适配器。主要实现位于：

模块	作用
<code>evidence-scoped-admission.ts</code>	来源、说话人和作用范围检查；混合主张投影
<code>final-draft-admission.ts</code>	审核 post-dedup 的最终写入草稿与完整证据窗口
<code>ll-extractor.ts</code>	将两层 admission 接入抽取、dedup 与写入
<code>recall-shadow-adapter.ts</code>	记录精确召回身份、预算和 prompt 暴露证据
<code>auto-recall.ts / memory-search.ts</code>	在两条 L1 读取路径统一应用精确干预
<code>tool-execution.ts</code>	将真实工具结果变成 task-bound observation
<code>execution-feedback.ts</code>	聚合 objective outcome 并定位首个证据断点
<code>openclaw-recall-turn-bridge.ts</code>	将召回、模型输入和任务结束事件绑定到同一 turn

全部新策略采用 opt-in 配置。关闭时继续执行原始 MemoryCore 路径。实验期间没有写入、删除或降权任何 production Memory。

4. 实验方法

4.1 数据分层

数据	规模	目的	结论边界
LongMemEval feedback-lifecycle	500 条；140 calibration、360 evaluation	检查词法反馈 baseline 的错误失效与漏检	离线生命周期实验，不评分端到端回答
TDAI/PAST EP02	1 个业务来源、4 个后续 case、两臂	接通 Qwen 抽取、TDAI 召回、工具 observation	系统链路开发验证
Wave Cedar	2 类目标 × 两臂 × 2 blocks	检查精确隔离、事实保留、主动搜索旁路	合成业务目标小样本
Vendor Boreal + Aurora Window	2 个新来源；4 次抽取、12 次目标调用	中英文新来源的端到端验收	固定本地模型和种子
mixed fact/behavior	14 个新模板	安全拆分、不可拆分和格式误报	手工反例组件集
r1 保存候选重放	2 个真实 Qwen 候选	检查新版对已发生错误的处理	只读重放，0 新模型调用
v4 最终草稿筛查	48 条、8 个语义组	disposition、validity、错误激活与保留	同窗口开发集

4.2 两臂控制

两臂使用相同模型、source 对话、工具能力和目标任务。普通臂保留原始抽取结果；调整臂启用 evidence-scoped admission 与精确读取干预。目标任务按 AB/BA 顺序运行，减少顺序偏差。规则在目标评分前固定，离线成绩不回流到当次处理。

4.3 指标口径

- **strict exact**：完整输出协议精确匹配，包括前缀、顺序和格式。
- **semantic facts**：只核对日期、ID、状态和范围等业务事实。
- **false activation**：临时或无来源支持的行为进入 active L1。
- **useful retention**：有来源支持的业务事实继续可用。
- **severe regression**：错误写入、意外修改等高风险副作用。
- **lifecycle tokens**：来源抽取与后续任务的 token 总和。

strict exact 和 semantic facts 分开，可以区分“弱模型漏写固定前缀”与“业务事实本身错误”。

5. 结果

5.1 Baseline 暴露出高误伤风险

LongMemEval 上，最初词法方案的 feedback precision 为 43.33%，recall 为 25.49%；它产生 34 次错误失效和 34 次 active miss。虽然 token proxy 下降 8.9%，但错误处理正确记忆会影响未来任务，因此该路径没有作为默认策略启用。

图 1 从词法处罚到来源锚定



两项指标来自不同实验，用于展示问题与改进方向，不构成同一排行榜。

5.2 本地弱模型端到端结果

模型为 Qwen3-4B-Instruct-2507-bf16。Vendor Boreal 为英文来源，Aurora Window 为中文来源；每个来源的目标任务按两种顺序重复。

指标	普通臂	调整臂	变化
strict exact	2/4	4/4	+2
semantic facts	4/4	4/4	持平
severe regressions	0	0	持平
target tokens	10,130	9,738	-392 (-3.87%)
lifecycle calls	8	8	持平
lifecycle tokens	13,995	13,578	-417 (-2.98%)
调整后活跃记忆无旧格式	—	2/2 来源	通过

普通臂的两次 Aurora 任务沿用了历史 WINDOW|...，没有执行当前要求的 MAINT|...。调整臂两次输出正确。四次回答的日期和起止时间都正确，差异集中在旧行为格式对当前输出的干扰。

该 exact 提升来自 instruction 范围审核。r2 新抽取的 episodic 较干净，新加入的混合主张投影没有在 r2 触发。两项能力的证据在下节分别报告。

5.3 混合主张处理

检查	原路径	Anchor
14 个模板动作判断	—	14/14
14 个模板最终内容	—	14/14
临时格式活跃残留	10	0
project / quarantine / unchanged	—	8 / 2 / 4
模型调用 / token	—	0 / 0
历史真实候选格式残留	1	0
历史真实候选事实保留	3/3	3/3

14 条结果覆盖中英文、混合语言、可拆分、不可拆分和格式误报。历史重放直接读取 r1 保存的 Qwen 候选及其引用来源，没有重新生成候选。

5.4 最终草稿开发筛查

v4 包含 48 条新合成候选，24 条应保留、24 条应隔离。旧词法 gate 的 disposition 为 27/48，validity 为 18/48，错误激活 15/24。当前本地路径达到 disposition 48/48、validity 46/48、错误激活 0/24、有用保留 24/24，在线模型调用为 0。

该数据与规则在同一开发窗口迭代，结果属于开发筛查。它说明最终草稿证据合同覆盖了旧 gate 的明显缺口，独立泛化仍需新来源验证。

6. 结果分析

6.1 已确认的收益

1. 范围错误可以在写入侧被阻断。Aurora 的两次输出差异表明，一次性格式进入 active instruction 会真实干扰后续行动。
2. 正确事实可以与临时行为分开。semantic facts 维持 4/4，r1 重放保留 3/3 事实。
3. 读取干预需要覆盖全部入口。Wave 首轮发现显式搜索旁路；统一排除后通过 2/2，泄漏降为 0。
4. 低成本规则适合处理窄而确定的错误。混合主张投影增加 0 次模型调用，减少了后续上下文。

6.2 尚未得到的结论

端到端来源只有两个，每臂四个目标结果；固定温度和种子产生的是一致性证据，独立样本数量有限。当前实验没有覆盖长期冲突、多来源更新、TTL、用户撤回组合及生产级权限。结果也没有建立 Anchor 与 Codex、Claude Code 的问题排行榜。

6.3 成本判断

调整臂目标 token 下降 3.87%，全生命周期下降 2.98%。省量来自移除无关行为上下文，没有额外同步模型税。Google、AWS、Mem0 等体系常将抽取、合并或反思放在后台模型任务中，能力上限更高，也会增加推理开销。Anchor 当前策略是同步路径处理确定项，复杂项进入异步队列；后续应报告升级率、GPU seconds per success 和 review queue 处理成本。

7. 与上游系统的设计对照

公开资料没有披露 Anthropic 或 OpenAI 的内部候选接纳与效用归因算法，因此本节比较公开机制和工程约束。

公开设计方向	代表系统	Anchor 的落实
项目/会话作用域与用户控制	Claude Memory、ChatGPT Memory/Projects	临时/长期范围检查；精确记录隔离；生产写回默认关闭
后台抽取、主题与 TTL	Google Vertex AI Memory Bank	同步确定性 gate；复杂语义预留异步 reviewer；TTL 待实现
偏好、事实、摘要、经历分层	AWS AgentCore Memory、LangGraph	L0 来源、episodic L1、active instruction 分层处理
episode、时间有效性与历史	Zep/Graphiti、Hindsight	来源可回查与 review queue；版本 lineage 和 freshness 待补
scope、异步写入与历史 API	Mem0	task-bound 信号、opt-in admission；持久 update/history 待补
常驻核心与按需历史	Letta/MemGPT	仅 active 行为常驻，大体量来源按需读取

Anchor 的价值在于把这些公开原则落到 TDAI Memory 的真实写入、召回、主动搜索和 Agent 执行链，并用本地弱模型量化质量与成本。上游公开机制提供设计背书，本地实验负责验证实际收益。

8. 开源交付与复现

代码以 TencentDB Agent Memory 的公开仓库为上游，沿用 MIT 许可证与原始版权声明。

- 项目主页：<https://github.com/newmancai/TencentDB-Agent-Memory/tree/feat/anchor-memory>
- 实现基准提交：<https://github.com/newmancai/TencentDB-Agent-Memory/commit/69486006e02800ad1a686399b4c0a6cfbf9acb70>
- Anchor 文档：`MemoryCore/research/anchor/`
- 核心代码：`MemoryCore/src/core/self-supervision/`
- 机器结果：`MemoryCore/research/anchor/results/`
- 固定实验脚本：`MemoryCore/research/anchor/scripts/`

在 MemoryCore 根目录运行：


```
npm install
npx vitest run
npm run build:plugin

node --import tsx \
  research/anchor/scripts/run_mixed_claim_projection_benchmark.ts \
  /tmp/anchor-mixed-claim-result.json
```

干净 fork 验证结果为 18 个测试文件、173 个测试全部通过；插件构建生成 6 个文件，约 1.22 MB。端到端脚本需要 PAST/Hermes 任务环境和本地 OpenAI-compatible 模型服务，仓库同时保存运行计划和已评分 JSON，便于无模型复核。

9. 下一阶段

下一轮优先增加至少 8 个真实抽取来源，覆盖撤回、事实更新、多 source、引用顺序和 TTL；同时更换模型、温度和种子。评测继续固定报告 strict exact、semantic facts、错误激活、事实保留、严重回归、unknown、tokens 和延迟。

工程侧需要补齐三项能力：持久版本替换与回滚；review queue 的异步处理和成本核算；user/project/session namespace 在存储与召回层的完整贯通。达到新来源验收后，再讨论 production shadow 部署和有限自动写回。

参考资料

1. Anthropic, [Claude memory and chat search](#).
2. Anthropic, [Claude Code power user tips](#).
3. OpenAI, [Memory FAQ](#).
4. OpenAI, [Projects in ChatGPT](#).
5. Google Cloud, [Configure Vertex AI Memory Bank](#).
6. AWS, [AgentCore Memory built-in strategies](#).
7. Mem0, [Asynchronous memory and memory update](#).
8. Zep, [Graphiti overview](#).
9. Hindsight, [Observations](#).
10. Letta, [Memory architecture](#).

附录：结论使用边界

本报告中的 LongMemEval、候选筛查、组件反例和端到端任务具有不同分母与用途。各项数字按原实验独立解释，没有拼接成统一总分。v2/v3 早期候选中存在诊断元数据泄漏，因此没有作为严格盲测证据；v4 仍属于同窗口开发集。完整运行记录和历史失败保存在研究归档，正文只呈现影响架构判断和结果解释的证据。